บทที่4

1. อธิบายความสำคัญของการจัดการเหตุการณ์จากผู้ใช้ต่อประสิทธิภาพของแอปพลิเคชัน หากไม่มีการจัดการที่ดีจะเกิดผลกระทบอะไรบ้าง พร้อมยกตัวอย่างโค้ดหรือสถานการณ์จริง

ตอบ การจัดการเหตุการณ์จากผู้ใช้ (User Event Handling) คือกระบวนการที่แอปพลิเคชันตอบสนองต่อการกระทำของผู้ใช้ เช่น การคลิก แตะ พิมพ์ เลื่อน หรือเลือกเมนู ถือเป็นจุดเชื่อมต่อสำคัญระหว่างผู้ใช้กับระบบ

ความสำคัญ

- ทำให้แอปพลิเคชันรับรู้ความต้องการของผู้ใช้และดำเนินการได้ถูกต้อง
- สร้างความลื่นไหลในการใช้งาน (Responsiveness)
- ลดความสับสนและข้อผิดพลาดในการใช้งาน

หากไม่มีการจัดการที่ดี จะเกิดผลกระทบ

- ผู้ใช้สับสน ไม่แน่ใจว่าระบบรับคำสั่งแล้วหรือไม่ (ไม่มี Feedback)
- กดปุ่มซ้ำหลายครั้งเพราะคิดว่าไม่ทำงาน อาจทำให้เกิดการทำรายการซ้ำ (เช่น สั่งซื้อสินค้าซ้ำ)
- แอปตอบสนองช้าหรือค้าง ทำให้ผู้ใช้เลิกใช้งาน
- ข้อมูลผิดพลาดเพราะไม่มีการตรวจสอบก่อนประมวลผล

ตัวอย่างสถานการณ์จริง

ปุ่ม "ชำระเงิน" ในแอปธนาคาร หากกดแล้วไม่มีการ disable ปุ่มหรือแสดง Loading ผู้ใช้อาจกดซ้ำหลายครั้ง ทำให้เกิดการตัดเงินซ้ำซ้อน

2. วิเคราะห์ข้อจำกัดของการรับข้อมูลผ่านแป้นพิมพ์ และเสนอแนวทางแก้ไข

ข้อจำกัด

- หน้าจอมือถือมีขนาดเล็ก แป้นพิมพ์เสมือน (Virtual Keyboard) จึงมีปุ่มเล็ก กดผิดง่าย
- ใช้เวลานานกว่าการแตะเลือก โดยเฉพาะข้อมูลซ้ำๆ เช่น วันที่ อีเมล
- เกิดข้อผิดพลาดจากการสะกดผิด/พิมพ์ผิด
- แป้นพิมพ์บังพื้นที่หน้าจอส่วนหนึ่ง ทำให้มองเห็นเนื้อหาอื่นได้น้อยลง
- ผู้ใช้บางกลุ่ม (ผู้สูงอายุ ผู้พิการทางสายตา) ใช้งานลำบาก

แนวทางแก้ไข

- ใช้ keyboardType ที่เหมาะกับข้อมูล เช่น ตัวเลข อีเมล เบอร์โทร เพื่อลดปุ่มที่ไม่จำเป็น
- ใช้ Auto-complete / Auto-suggest ช่วยเดาคำ
- ใช้ Dropdown, Date Picker, Radio Button แทนการพิมพ์เมื่อเป็นไปได้
- มี Client-side Validation ตรวจสอบทันทีเมื่อพิมพ์ผิดรูปแบบ
- รองรับ Voice Input เป็นทางเลือกเสริม

3. อธิบายข้อดีและข้อจำกัดของการรับข้อมูลผ่านเมาส์ พร้อมยกตัวอย่างการใช้งานจริง

ข้อดี

- แม่นยำสูง เหมาะกับงานละเอียด เช่น การวาดภาพ ตัดต่อวิดีโอ ออกแบบกราฟิก
- รองรับการทำงานหลายรูปแบบ (คลิกซ้าย-ขวา, ดับเบิลคลิก, ลาก-วาง, scroll wheel)
- ใช้ร่วมกับคีย์บอร์ดได้สะดวกในการทำงานที่ซับซ้อน เช่น การเลือกหลายรายการพร้อมกัน

ข้อจำกัด

- ใช้ไม่ได้กับอุปกรณ์พกพา (สมาร์ตโฟน/แท็บเล็ต) เพราะไม่มีพอร์ตหรือพื้นที่วาง
- ต้องมีพื้นผิวรองรับการเลื่อน (mouse pad) ไม่สะดวกเมื่อเคลื่อนที่
- ผู้ใช้ที่มีปัญหาด้านการเคลื่อนไหวมือ อาจควบคุมยาก

ตัวอย่างการใช้งานจริง

โปรแกรม Photoshop หรือ CAD ที่ต้องการความแม่นยำระดับพิกเซลในการเลือกพื้นที่หรือวาดเส้น

4. อธิบายข้อดีและข้อจำกัดของการรับข้อมูลผ่านหน้าจอสัมผัส พร้อมยกตัวอย่างการใช้งานจริง

ข้อดี

- ใช้งานง่าย เป็นธรรมชาติ (แตะ ลาก ปัด บีบขยาย) ไม่ต้องใช้อุปกรณ์เสริม
- เหมาะกับอุปกรณ์พกพา ใช้งานได้ทุกที่ทุกเวลา
- รองรับ Multi-touch เช่น การซูมภาพด้วยสองนิ้ว

ข้อจำกัด

- ความแม่นยำต่ำกว่าเมาส์ นิ้วมือบังพื้นที่ที่กำลังกด (Fat Finger Problem)
- ไม่เหมาะกับงานที่ต้องการความละเอียดสูง เช่น การแก้ไขข้อความจำนวนมาก
- หน้าจอเปื้อนรอยนิ้วมือ และเสื่อมสภาพจากการสัมผัสบ่อย
- ใช้งานลำบากเมื่อมือเปียกหรือสวมถุงมือ

ตัวอย่างการใช้งานจริง

แอปแผนที่ (Google Maps) ที่ใช้การปัดเลื่อนแผนที่และบีบนิ้วเพื่อซูมเข้า-ออก

5. อธิบายข้อดีและข้อจำกัดของการรับข้อมูลผ่านเสียง พร้อมยกตัวอย่างการใช้งานจริง

ข้อดี

- ใช้งานได้แบบไม่ต้องใช้มือ (Hands-free) เหมาะกับการขับรถ หรือทำกิจกรรมอื่นพร้อมกัน
- ช่วยเหลือผู้พิการทางการเคลื่อนไหวหรือทางสายตา
- รวดเร็วกว่าการพิมพ์ในบางกรณี เช่น การสั่งงานสั้นๆ

ข้อจำกัด

- ความแม่นยำขึ้นอยู่กับสำเนียง ภาษา และเสียงรบกวนรอบข้าง
- มีปัญหาด้านความเป็นส่วนตัวเมื่อใช้ในที่สาธารณะ

- ต้องเชื่อมต่ออินเทอร์เน็ตในหลายกรณี (ระบบประมวลผลบน Cloud)
- ไม่เหมาะกับข้อมูลที่ซับซ้อนหรือมีรายละเอียดมาก

ตัวอย่างการใช้งานจริง

ผู้ช่วยเสียง เช่น Siri, Google Assistant ใช้สั่งเปิดเพลง ตั้งนาฬิกาปลุก หรือส่งข้อความด้วยเสียง

6. อธิบายความแตกต่างระหว่าง Notification และ Dialog ในการสื่อสารกับผู้ใช้ พร้อมยกตัวอย่างการใช้งานจริง

หัวข้อ	Notification	Dialog
ลักษณะ	แจ้งเตือนแบบไม่รบกวนงานหลัก	หน้าต่างที่บังคับให้ผู้ใช้ตอบสนองก่อนทำงานอื่นต่อ
การโต้ตอบ	ไม่จำเป็นต้องตอบสนองทันที	ต้องกดปุ่มยืนยัน/ยกเลิกก่อนถึงจะใช้งานส่วนอื่นได้
ตำแหน่ง	แสดงนอกแอป หรือมุมหน้าจอ	แสดงตรงกลางหน้าจอ ทับเนื้อหาหลัก
ตัวอย่าง	แจ้งเตือนข้อความใหม่จากแอป LINE	หน้าต่างยืนยัน "คุณต้องการลบไฟล์นี้หรือไม่?"

โดยสรุป Notification เหมาะกับข้อมูลที่ผู้ใช้ควรทราบแต่ไม่จำเป็นต้องหยุดสิ่งที่ทำอยู่ ส่วน Dialog ใช้เมื่อจำเป็นต้องได้รับการตัดสินใจหรือยืนยันจากผู้ใช้ก่อนดำเนินการต่อ เช่น การลบข้อมูลสำคัญ

7. อธิบายการใช้ Notification แบบชั่วคราว (Snackbar/Toast) และผลกระทบต่อประสบการณ์ผู้ใช้

ตอบ Snackbar/Toast คือข้อความแจ้งเตือนขนาดเล็กที่ปรากฏชั่วคราวบริเวณด้านล่างหรือบางส่วนของหน้าจอ แล้วหายไปเองภายในไม่กี่วินาที โดยไม่บังคับให้ผู้ใช้กดยืนยัน สอดคล้องกับหลักการ "มีการแจ้งผลลัพธ์ (Feedback)"

ผลกระทบต่อประสบการณ์ผู้ใช้ (เชิงบวก)

- แจ้งผลการทำงาน (สำเร็จ/ล้มเหลว) โดยไม่ขัดจังหวะการใช้งาน สอดคล้องกับหลัก "ตอบสนองรวดเร็ว"
- ลดความสับสน ทำให้ผู้ใช้มั่นใจว่าคำสั่งถูกดำเนินการแล้ว

- ไม่รบกวนการทำงานต่อเนื่อง เพราะหายไปเองโดยไม่ต้องปิดเอง

ผลกระทบเชิงลบ (หากใช้ไม่เหมาะสม)

- หากข้อความสำคัญมาก แต่ Snackbar หายเร็วเกินไป ผู้ใช้อาจพลาดข้อมูล
- ใช้บ่อยเกินไปอาจสร้างความรำคาญ

8. อธิบายข้อดีของการตรวจสอบข้อมูลฝั่งผู้ใช้ (Client-side Validation) และผลกระทบต่อประสิทธิภาพระบบ

ตอบ Client-side Validation คือการตรวจสอบความถูกต้องของข้อมูลบนอุปกรณ์ของผู้ใช้ก่อนส่งไปยังเซิร์ฟเวอร์ สอดคล้องกับหลักการ "ป้องกันข้อผิดพลาด"

ข้อดี

- **ตอบสนองทันที** ผู้ใช้รู้ผลทันทีโดยไม่ต้องรอเชื่อมต่อเซิร์ฟเวอร์ ทำให้ประสบการณ์ใช้งานลื่นไหล
- **ลดภาระเซิร์ฟเวอร์** เพราะกรองข้อมูลผิดพลาดออกไปก่อนส่ง ลดจำนวน Request ที่ไม่จำเป็น
- **ลดการใช้แบนด์วิดท์** ไม่ต้องส่ง-รับข้อมูลผิดพลาดไปกลับ
- **ป้องกันข้อผิดพลาดเบื้องต้น** เช่น กรอกอีเมลผิดรูปแบบ หรือเว้นช่องที่จำเป็นว่างไว้

ผลกระทบต่อประสิทธิภาพระบบ

ทำให้ระบบโดยรวมทำงานเร็วขึ้น เพราะลดการประมวลผลที่ไม่จำเป็นบนฝั่งเซิร์ฟเวอร์ อย่างไรก็ตาม Client-side Validation ไม่ควรใช้แทน Server-side Validation ทั้งหมด เพราะสามารถถูก bypass ได้ (เช่น ผ่าน API โดยตรง) จึงควรใช้ควบคู่กันเพื่อความปลอดภัยของระบบ

บทที่5

1. อธิบายหลักการออกแบบฟอร์มที่ดี และผลกระทบต่อประสบการณ์ผู้ใช้

ตอบ การออกแบบฟอร์ม (Form Design) ที่ดีเป็นปัจจัยสำคัญที่ช่วยให้ผู้ใช้กรอกข้อมูลได้อย่างถูกต้อง รวดเร็ว และไม่เกิดความสับสน โดยมีหลักการสำคัญดังนี้

- เรียบง่ายและชัดเจน (Simplicity & Clarity): ใช้จำนวนช่องกรอกข้อมูลเท่าที่จำเป็น ไม่รกเกินไป
- จัดลำดับข้อมูลอย่างเป็นระบบ: เรียงลำดับจากข้อมูลทั่วไปไปสู่ข้อมูลเฉพาะ เช่น ชื่อ–ที่อยู่–ข้อมูลติดต่อ
- มีป้ายกำกับ (Label) ที่ชัดเจน: บอกผู้ใช้ว่าควรกรอกข้อมูลอะไรในแต่ละช่อง
- ใช้ตัวอย่างหรือคำแนะนำ (Placeholder/Hint): ช่วยลดความสับสนในการกรอกข้อมูล
- รองรับการใช้งานบนอุปกรณ์หลากหลาย: เช่น คอมพิวเตอร์ แท็บเล็ต และสมาร์ตโฟน
- มีการตรวจสอบข้อมูล (Validation) และแจ้งข้อผิดพลาดทันทีที่ผู้ใช้กรอกผิด

ผลกระทบต่อประสบการณ์ผู้ใช้: หากออกแบบฟอร์มได้ดี จะช่วยลดข้อผิดพลาดในการกรอกข้อมูล ลดเวลาที่ใช้ในการกรอกฟอร์ม เพิ่มความมั่นใจและความพึงพอใจของผู้ใช้ ในทางกลับกัน หากฟอร์มมีความซับซ้อน มีขั้นตอนมากเกินไป หรือไม่มีคำอธิบายที่ชัดเจน ผู้ใช้อาจกรอกข้อมูลผิดพลาด เกิดความหงุดหงิด และอาจเลิกใช้งานแอปพลิเคชันไปในที่สุด

2. ยกตัวอย่างการใช้งานฟอร์มเข้าสู่ระบบ (Login Form) และอธิบายความสำคัญในการตรวจสอบตัวตน

ตอบ ฟอร์มเข้าสู่ระบบ (Login Form) เป็นฟอร์มประเภทหนึ่งที่ใช้ตรวจสอบตัวตนของผู้ใช้ ประกอบด้วยช่องกรอกข้อมูลหลัก เช่น ชื่อผู้ใช้หรืออีเมล (Username/Email) และรหัสผ่าน (Password) พร้อมปุ่มเข้าสู่ระบบ ตัวอย่างการทำงาน เช่น

- ผู้ใช้กรอกอีเมลและรหัสผ่านลงในฟอร์ม
- ระบบตรวจสอบรูปแบบข้อมูลเบื้องต้นฝั่งผู้ใช้ (Client-side Validation) เช่น ตรวจสอบว่าช่องข้อมูลว่างหรือไม่ รูปแบบอีเมลถูกต้องหรือไม่
- เมื่อกดปุ่มเข้าสู่ระบบ ข้อมูลจะถูกส่งไปตรวจสอบกับฐานข้อมูลฝั่งเซิร์ฟเวอร์ (Server-side Validation) เพื่อยืนยันว่าชื่อผู้ใช้และรหัสผ่านตรงกับที่บันทึกไว้

- หากข้อมูลถูกต้อง ระบบจะอนุญาตให้เข้าสู่ระบบ แต่หากไม่ถูกต้อง ระบบจะแสดงข้อความแจ้งเตือนข้อผิดพลาด

ความสำคัญในการตรวจสอบตัวตน (Authentication):
การตรวจสอบตัวตนมีความสำคัญอย่างมากต่อความปลอดภัยของระบบ เพราะช่วยป้องกันไม่ให้บุคคลที่ไม่ได้รับอนุญาตเข้าถึงข้อมูลส่วนตัวหรือทรัพยากรของผู้ใช้ ป้องกันการปลอมแปลงตัวตน (Identity Theft) และช่วยรักษาความน่าเชื่อถือของระบบ นอกจากนี้ยังเป็นด่านแรกในการควบคุมสิทธิ์การเข้าถึงข้อมูล (Authorization) ในขั้นตอนถัดไปของแอปพลิเคชันอีกด้วย

3. วิเคราะห์ข้อจำกัดของการจัดเก็บข้อมูลแบบชั่วคราว และเสนอแนวทางแก้ไข

ตอบ การจัดเก็บข้อมูลแบบชั่วคราว (Temporary Storage)
คือการเก็บข้อมูลไว้ในหน่วยความจำของโปรแกรมขณะที่แอปพลิเคชันกำลังทำงานอยู่ เช่น การเก็บค่าไว้ในตัวแปรระหว่างการทำงาน (In-memory Storage) ซึ่งมีข้อจำกัดดังนี้

- ข้อมูลจะหายไปทันทีเมื่อปิดแอปพลิเคชัน รีสตาร์ทเครื่อง หรือแอปพลิเคชันหยุดทำงานกะทันหัน
- ไม่สามารถแชร์ข้อมูลข้ามการเปิดใช้งานแต่ละครั้งได้ ผู้ใช้ต้องกรอกข้อมูลใหม่ทุกครั้ง
- ไม่เหมาะกับข้อมูลที่ต้องการความคงอยู่ระยะยาว เช่น ประวัติการใช้งาน การตั้งค่าผู้ใช้ หรือบัญชีผู้ใช้
- มีความเสี่ยงต่อการสูญหายของข้อมูลหากเกิดข้อผิดพลาดของระบบ (Crash) ระหว่างการทำงาน

แนวทางแก้ไข: ควรจัดเก็บข้อมูลที่จำเป็นต้องคงอยู่ลงในหน่วยเก็บข้อมูลแบบถาวร (Persistent Storage) แทน เช่น การใช้ Local Storage หรือฐานข้อมูลภายในเครื่อง (เช่น SharedPreferences, SQLite ในแอป Flutter หรือ TinyDB ใน MIT App Inventor) หรือจัดเก็บข้อมูลไว้บนระบบฐานข้อมูลออนไลน์ (Cloud Database) เพื่อให้ข้อมูลยังคงอยู่แม้ปิดแอปพลิเคชันหรือเปลี่ยนอุปกรณ์

4. อธิบายการใช้หน่วยความจำชั่วคราว และผลกระทบเมื่อระบบหยุดทำงานหรือปิดเครื่อง

ตอบ หน่วยความจำชั่วคราว (RAM หรือ Volatile Memory)
เป็นพื้นที่ที่โปรแกรมใช้เก็บค่าตัวแปรและข้อมูลต่าง ๆ ในระหว่างที่แอปพลิเคชันกำลังทำงาน เช่น

ค่าที่ผู้ใช้กรอกในฟอร์ม ค่าที่คำนวณได้ระหว่างการประมวลผล หรือสถานะปัจจุบันของหน้าจอ
ข้อมูลเหล่านี้จะถูกเก็บไว้เฉพาะช่วงเวลาที่โปรแกรมกำลังทำงานอยู่เท่านั้น
ผลกระทบเมื่อระบบหยุดทำงานหรือปิดเครื่อง: เมื่อแอปพลิเคชันถูกปิด ถูกบังคับปิด (Force Close)
หรือเครื่องถูกปิดเครื่อง/รีสตาร์ท ข้อมูลทั้งหมดที่อยู่ในหน่วยความจำชั่วคราวจะถูกลบทิ้งทันที
เนื่องจากหน่วยความจำประเภทนี้ต้องอาศัยกระแสไฟฟ้าในการรักษาข้อมูลไว้
ดังนั้นหากนักพัฒนาต้องการให้ข้อมูลสำคัญยังคงอยู่หลังจากปิดแอปพลิเคชัน
จำเป็นต้องบันทึกข้อมูลลงในหน่วยเก็บข้อมูลถาวร เช่น ไฟล์ ฐานข้อมูลภายในเครื่อง
หรือฐานข้อมูลออนไลน์ ก่อนที่แอปพลิเคชันจะถูกปิด

5. อธิบายการใช้ตัวแปรแบบโกลบอล และผลกระทบต่อการทำงานของโปรแกรม

ตอบ ตัวแปรแบบโกลบอล (Global Variable) คือตัวแปรที่ถูกประกาศไว้นอกฟังก์ชันหรือคลาส
ทำให้สามารถเข้าถึงและใช้งานได้จากทุกส่วนของโปรแกรม ไม่จำกัดอยู่เฉพาะในฟังก์ชันใดฟังก์ชันหนึ่ง
เหมาะสำหรับการเก็บค่าที่ต้องใช้ร่วมกันในหลายหน้าจอ เช่น ข้อมูลผู้ใช้ที่ล็อกอินอยู่
หรือค่าการตั้งค่าของแอปพลิเคชัน
ผลกระทบต่อการทำงานของโปรแกรม:

- ข้อดี: สะดวกในการแชร์ข้อมูลระหว่างหน้าจอหรือฟังก์ชันต่าง ๆ โดยไม่ต้องส่งค่าผ่านพารามิเตอร์
- ข้อเสีย: หากมีหลายส่วนของโปรแกรมเข้าถึงและแก้ไขตัวแปรเดียวกันพร้อมกัน
  อาจทำให้เกิดข้อผิดพลาดที่ตรวจสอบและแก้ไขได้ยาก (Side Effect)
- ทำให้โครงสร้างโปรแกรมซับซ้อน ยากต่อการดูแลรักษาและทดสอบ (Testing) ในระยะยาว
- มีความเสี่ยงที่ชื่อตัวแปรจะซ้ำกันหรือถูกเปลี่ยนแปลงโดยไม่ตั้งใจจากส่วนอื่นของโปรแกรม

ดังนั้นควรใช้ตัวแปรโกลบอลเท่าที่จำเป็น และพิจารณาใช้แนวทางการจัดการสถานะ (State
Management) ที่เหมาะสมแทน เพื่อให้โปรแกรมมีโครงสร้างที่ดีและง่ายต่อการบำรุงรักษา

6. อธิบายโครงสร้างข้อมูลที่ใช้เก็บหลายค่าในตัวแปรเดียว พร้อมยกตัวอย่างการใช้งานจริง

ตอบ โครงสร้างข้อมูลที่ใช้เก็บหลายค่าไว้ในตัวแปรเดียวที่นิยมใช้ คือ อาร์เรย์ (Array) หรือลิสต์ (List) ซึ่งสามารถเก็บข้อมูลชนิดเดียวกันได้หลายค่าเรียงต่อกันในตัวแปรเดียว โดยแต่ละค่าจะมีลำดับ (Index) กำกับ ทำให้สามารถเรียกใช้งานค่าที่ต้องการได้อย่างสะดวก

ตัวอย่างการใช้งานจริงในภาษา Dart (Flutter):

```
List<String> fruits = ['แอปเปิ้ล', 'กล้วย', 'ส้ม'];


print(fruits[0]);   // แสดงผล: แอปเปิ้ล
fruits.add('มะม่วง'); // เพิ่มข้อมูลใหม่เข้าไปในลิสต์
```

ตัวอย่างการใช้งานจริง เช่น การเก็บรายการสินค้าในตะกร้าสินค้า (Shopping Cart),

การเก็บรายชื่อผู้ใช้ในระบบแชท, หรือการเก็บรายการงานในแอปพลิเคชัน To-Do List

ซึ่งช่วยให้จัดการข้อมูลจำนวนมากได้อย่างเป็นระบบ แทนที่จะต้องประกาศตัวแปรแยกทีละตัว

7. อธิบายการจัดเก็บข้อมูลภายในอุปกรณ์
และผลต่อการทำงานของแอปพลิเคชันเมื่อไม่มีอินเทอร์เน็ต

ตอบ การจัดเก็บข้อมูลภายในอุปกรณ์ (On-device / Local Storage)
คือการบันทึกข้อมูลไว้ในหน่วยความจำถาวรของอุปกรณ์ เช่น หน่วยความจำภายในเครื่อง (Internal Storage) แทนที่จะเก็บไว้บนเซิร์ฟเวอร์หรือฐานข้อมูลออนไลน์เพียงอย่างเดียว ตัวอย่างวิธีการจัดเก็บได้แก่ การเก็บเป็นไฟล์ การใช้ฐานข้อมูล SQLite หรือการใช้ Key-Value Storage เช่น SharedPreferences

ผลต่อการทำงานของแอปพลิเคชันเมื่อไม่มีอินเทอร์เน็ต:

หากแอปพลิเคชันมีการจัดเก็บข้อมูลภายในอุปกรณ์ไว้แล้ว

ผู้ใช้ยังคงสามารถเปิดใช้งานแอปพลิเคชันและเข้าถึงข้อมูลที่เคยบันทึกไว้ได้

แม้ไม่มีการเชื่อมต่ออินเทอร์เน็ต (Offline Mode) เช่น ดูข้อมูลที่เคยโหลดไว้ก่อนหน้า

หรือกรอกข้อมูลค้างไว้แล้วค่อยซิงค์กับเซิร์ฟเวอร์ภายหลังเมื่อกลับมามีสัญญาณอินเทอร์เน็ต

ในทางตรงกันข้าม

หากแอปพลิเคชันพึ่งพาการเชื่อมต่อออนไลน์เพียงอย่างเดียวโดยไม่มีการจัดเก็บข้อมูลภายในเครื่อง แอปพลิเคชันจะไม่สามารถใช้งานได้เลยเมื่อไม่มีอินเทอร์เน็ต

8. อธิบายการใช้ Local Storage ในแอปพลิเคชัน พร้อมยกตัวอย่างการใช้งานจริง

ตอบ Local Storage คือรูปแบบการจัดเก็บข้อมูลแบบ Key-Value ไว้ภายในอุปกรณ์ของผู้ใช้ เหมาะสำหรับเก็บข้อมูลขนาดเล็กที่ไม่ซับซ้อน เช่น การตั้งค่าของแอปพลิเคชัน สถานะการเข้าสู่ระบบ หรือค่าที่ผู้ใช้เลือกไว้ล่าสุด ในการพัฒนาแอปพลิเคชันด้วย Flutter นิยมใช้แพ็กเกจ SharedPreferences ในการจัดการ Local Storage

ตัวอย่างการใช้งานจริง:

```
// บันทึกค่าลง Local Storage
final prefs = await SharedPreferences.getInstance();
await prefs.setString('username', 'somchai');
await prefs.setBool('isDarkMode', true);

// อ่านค่าจาก Local Storage
String? username = prefs.getString('username');
```

ตัวอย่างการใช้งานจริง เช่น การจดจำว่าผู้ใช้เคยล็อกอินไว้แล้วเพื่อไม่ต้องกรอกรหัสผ่านซ้ำทุกครั้ง การบันทึกธีมสี (โหมดสว่าง/มืด) ที่ผู้ใช้เลือกไว้ หรือการเก็บค่าภาษาที่ผู้ใช้เลือกใช้งานในแอปพลิเคชัน

9. อธิบายการใช้ TinyDB ใน MIT App Inventor สำหรับการจัดเก็บข้อมูลภายใน

ตอบ TinyDB เป็นคอมโพเนนต์ (Component) ใน MIT App Inventor ที่ใช้สำหรับจัดเก็บข้อมูลแบบถาวร (Persistent Storage) ไว้ภายในเครื่องของผู้ใช้

โดยข้อมูลจะยังคงอยู่แม้ปิดแอปพลิเคชันหรือปิดเครื่องไปแล้ว ทำงานในลักษณะ Key-Value คล้ายกับ Local Storage โดยมีการใช้งานหลักผ่านบล็อกคำสั่งดังนี้

- StoreValue: ใช้สำหรับบันทึกค่าข้อมูล โดยระบุ Tag (ชื่อกำกับข้อมูล) และ ValueToStore (ค่าที่ต้องการบันทึก)
- GetValue: ใช้สำหรับดึงค่าข้อมูลที่เคยบันทึกไว้ โดยระบุ Tag ที่ต้องการ และสามารถกำหนดค่าเริ่มต้น (ValueIfTagNotThere) กรณีไม่พบข้อมูล
- ClearTag: ใช้สำหรับลบข้อมูลเฉพาะ Tag ที่ระบุ
- ClearAll: ใช้สำหรับลบข้อมูลทั้งหมดที่เก็บไว้ใน TinyDB

ตัวอย่างการใช้งานจริง เช่น การบันทึกคะแนนสูงสุด (High Score) ของเกมไว้ในเครื่อง เพื่อให้ผู้เล่นเห็นคะแนนสูงสุดของตนเองทุกครั้งที่เปิดแอปพลิเคชัน การบันทึกรายการ To-Do List ของผู้ใช้ หรือการจดจำการตั้งค่าต่าง ๆ ของผู้ใช้ในแอปพลิเคชันที่พัฒนาด้วย MIT App Inventor โดยไม่จำเป็นต้องเชื่อมต่อกับฐานข้อมูลออนไลน์

บทที่6

1. อธิบายหลักการออกแบบ UI โดยใช้ White Space และผลกระทบต่อประสบการณ์ผู้ใช้

ตอบ White Space (พื้นที่ว่าง) หมายถึง พื้นที่ที่ไม่มีการแสดงองค์ประกอบใด ๆ บนหน้าจอ เช่น ช่องว่างระหว่างข้อความ ระหว่างปุ่ม หรือระหว่างรูปภาพกับขอบจอ ซึ่งถือเป็นหลักการออกแบบ UI ที่สำคัญ เพราะช่วยให้ผู้ใช้สามารถแยกแยะองค์ประกอบต่าง ๆ ออกจากกันได้อย่างชัดเจน ลดความรกและความสับสนของหน้าจอ ทำให้เนื้อหาที่สำคัญโดดเด่นและอ่านง่ายขึ้น

ในการพัฒนาแอปพลิเคชันด้วย Flutter สามารถกำหนด White Space ได้ผ่านพารามิเตอร์ต่าง ๆ เช่น padding และ margin ของ Container หรือการใช้วิดเจ็ต SizedBox เพื่อเว้นระยะห่างระหว่างวิดเจ็ต

ผลกระทบต่อประสบการณ์ผู้ใช้ (User Experience)

- หากมี White Space ที่เหมาะสม จะช่วยให้ UI ดูเป็นระเบียบ สบายตา และใช้งานง่าย ผู้ใช้สามารถโฟกัสไปยังเนื้อหาหรือปุ่มที่ต้องการกดได้ถูกต้อง
- หากมี White Space น้อยเกินไป องค์ประกอบต่าง ๆ จะอยู่ชิดกันจนดูรก ผู้ใช้อาจกดปุ่มผิดหรืออ่านเนื้อหาลำบาก
- หากมี White Space มากเกินไป อาจทำให้หน้าจอดูโล่งเกินไป เสียพื้นที่ในการแสดงเนื้อหา และผู้ใช้ต้องเลื่อนหน้าจอมากขึ้น

2. อธิบายการใช้ Column Widget ในการจัดเรียงวิดเจ็ต พร้อมยกตัวอย่างการใช้งานจริง

ตอบ Column เป็นวิดเจ็ตหลักที่ใช้สำหรับจัดเรียงวิดเจ็ตย่อย ๆ ในแนวตั้ง (Vertical) เรียงต่อกันตามลำดับจากบนลงล่าง โดยแนวแกนหลัก (Main Axis) ของ Column จะอยู่ในแนวตั้ง และแนวแกนขวาง (Cross Axis) จะอยู่ในแนวนอน Column สามารถกำหนดพารามิเตอร์ children เพื่อบรรจุวิดเจ็ตย่อยได้หลายตัว รวมถึงกำหนดการจัดเรียงด้วย mainAxisAlignment และ crossAxisAlignment ได้

```
Column(
  mainAxisAlignment: MainAxisAlignment.xxx,
  crossAxisAlignment: CrossAxisAlignment.xxx,
  children: [ ... ]
)
```

ตัวอย่างการใช้งานจริง: การจัดวางหน้าโปรไฟล์ผู้ใช้ที่ต้องแสดงรูปโปรไฟล์ ชื่อผู้ใช้ และคำอธิบายเรียงกันในแนวตั้ง หรือการจัดวางฟอร์มกรอกข้อมูลที่มีช่องกรอกชื่อ อีเมล และรหัสผ่านเรียงต่อกันจากบนลงล่าง

3. อธิบายความหมายของ MainAxisAlignment.spaceEvenly และ CrossAxisAlignment.stretch ใน Column Widget พร้อมยกตัวอย่างการใช้งานจริง

ตอบ MainAxisAlignment.spaceEvenly เป็นการกำหนดการจัดเรียงวิดเจ็ตตามแนวแกนหลัก (Main Axis) ของ Column คือแนวตั้ง โดยจะให้มีช่องว่างระหว่างวิดเจ็ตแต่ละอันเป็นระยะเท่า ๆ กันทั้งหมด รวมถึงช่องว่างก่อนวิดเจ็ตแรกและหลังวิดเจ็ตสุดท้ายด้วย ทำให้วิดเจ็ตทุกตัวกระจายตัวสม่ำเสมอเต็มพื้นที่ของ Column

CrossAxisAlignment.stretch เป็นการกำหนดการจัดเรียงตามแนวแกนขวาง (Cross Axis) ของ Column คือแนวนอน โดยจะขยายขนาดของวิดเจ็ตลูกทุกตัวให้เต็มความกว้างของแนวแกนขวาง (เต็มความกว้างของ Column)

ตัวอย่างการใช้งานจริง: การออกแบบหน้าเมนูที่มีปุ่มหลายปุ่มเรียงในแนวตั้ง โดยใช้ spaceEvenly เพื่อให้ระยะห่างระหว่างปุ่มแต่ละปุ่มเท่ากันสวยงาม และใช้ CrossAxisAlignment.stretch เพื่อให้ทุกปุ่มมีความกว้างเท่ากับ Column พอดี ไม่ต้องมากำหนดความกว้างของแต่ละปุ่มเอง

4. อธิบายความหมายของ CrossAxisAlignment.end และผลต่อการจัดวางวิดเจ็ต

ตอบ CrossAxisAlignment.end คือการกำหนดให้วิดเจ็ตลูกทั้งหมดถูกจัดวางชิดกับขอบด้านท้าย (End) ของแนวแกนขวาง (Cross Axis) เช่น ถ้าใช้ใน Column ซึ่งแนวแกนขวางคือแนวนอน วิดเจ็ตทุกตัวจะถูกจัดให้ชิดขอบด้านขวาของหน้าจอ หรือถ้าใช้ใน Row ซึ่งแนวแกนขวางคือแนวตั้ง วิดเจ็ตจะถูกจัดให้ชิดขอบด้านล่าง

ผลต่อการจัดวางวิดเจ็ต: ทำให้เกิดการจัดตำแหน่งที่ไม่สมมาตรกับด้านตรงข้าม (start) เหมาะสำหรับกรณีที่ต้องการเน้นให้องค์ประกอบอยู่ชิดด้านใดด้านหนึ่งโดยเฉพาะ เช่น การจัดวางปุ่ม "ถัดไป" ให้ชิดขอบขวาของหน้าจอ ในขณะที่เนื้อหาอื่น ๆ อาจอยู่ตำแหน่งปกติ

5. อธิบายการใช้ Feedback Components เช่น Progress Bar ในการแสดงความคืบหน้า

ตอบFeedback Components คือกลุ่มวิดเจ็ตที่ใช้สื่อสารสถานะการทำงานของระบบให้ผู้ใช้รับทราบ เพื่อให้ผู้ใช้เข้าใจว่าขณะนี้แอปพลิเคชันกำลังทำงานอยู่หรือไม่ และต้องรออีกนานเท่าใด ตัวอย่างที่พบบ่อยคือ Progress Bar (แถบแสดงความคืบหน้า) ซึ่งในเฟรมเวิร์ก Flutter จะมีวิดเจ็ตหลักคือ LinearProgressIndicator (แสดงเป็นแถบยาวในแนวนอน) และ CircularProgressIndicator (แสดงเป็นวงกลมหมุน)

Progress Bar มีสองรูปแบบการทำงานหลัก คือ แบบกำหนดค่าความคืบหน้า (Determinate) ซึ่งจะแสดงเปอร์เซ็นต์ความคืบหน้าที่แท้จริง เช่น การอัปโหลดไฟล์ที่ทราบขนาดข้อมูลแน่นอน และแบบไม่กำหนดค่า (Indeterminate) ซึ่งจะเคลื่อนไหวหรือหมุนไปเรื่อย ๆ เพื่อบอกว่าระบบกำลังทำงานอยู่ โดยไม่ทราบระยะเวลาที่แน่นอน เช่น การรอโหลดข้อมูลจากเซิร์ฟเวอร์

ตัวอย่างการใช้งานจริง: การแสดง CircularProgressIndicator ขณะรอข้อมูลจาก API หรือการแสดง LinearProgressIndicator ขณะอัปโหลดรูปภาพ เพื่อให้ผู้ใช้ทราบว่าระบบยังไม่หยุดทำงานหรือค้าง ช่วยลดความสับสนและเพิ่มความมั่นใจในการใช้งานแอปพลิเคชัน

6. อธิบายการใช้ BoxShape.circle ใน Container และผลลัพธ์ที่ได้

ตอบ BoxShape.circle เป็นค่าที่กำหนดให้แก่พารามิเตอร์ shape ภายใน BoxDecoration ของ Container เพื่อกำหนดรูปทรงของ Container ให้เป็นวงกลมแทนที่จะเป็นสี่เหลี่ยมแบบปกติ (ค่าเริ่มต้นคือ BoxShape.rectangle)

```
Container(
  decoration: BoxDecoration(
    color: Colors.black12,
    shape: BoxShape.circle,
  ),
  child: Text('Text'),
)
```

ผลลัพธ์ที่ได้: Container จะถูกวาดเป็นรูปวงกลมโดยอัตโนมัติตามขนาดที่กำหนด (width และ height ควรมีค่าเท่ากันเพื่อให้ได้วงกลมสมบูรณ์) ทั้งนี้เมื่อเลือกใช้ shape แล้ว จะไม่สามารถกำหนดพารามิเตอร์ borderRadius ได้อีก เนื่องจากทั้งสองพารามิเตอร์ใช้เพื่อกำหนดรูปทรงของ Container เหมือนกัน จึงต้องเลือกใช้อย่างใดอย่างหนึ่งเท่านั้น ตัวอย่างการใช้งานจริงคือการสร้างรูปโปรไฟล์ผู้ใช้ทรงวงกลม (Avatar) หรือไอคอนวงกลมสำหรับปุ่มลอย (Floating Action Button)

7. อธิบายการเปลี่ยน scrollDirection เป็น Axis.horizontal และผลต่อการเลื่อนหน้าจอ

ตอบ

scrollDirection เป็นพารามิเตอร์ของวิดเจ็ตที่รองรับการเลื่อนหน้าจอ เช่น SingleChildScrollView หรือ ListView ใช้สำหรับกำหนดทิศทางที่ผู้ใช้สามารถเลื่อน (scroll) เนื้อหาได้ ซึ่งค่าเริ่มต้นคือ Axis.vertical หมายถึงเลื่อนขึ้นลงในแนวตั้ง

เมื่อเปลี่ยนค่าเป็น Axis.horizontal จะทำให้ทิศทางการเลื่อนเปลี่ยนจากแนวตั้งเป็นแนวนอนแทน คือผู้ใช้จะเลื่อนหน้าจอไปทางซ้าย-ขวาแทนการเลื่อนขึ้น-ลง ซึ่งเหมาะสำหรับกรณีที่มีวิดเจ็ตย่อยจำนวนมากเรียงกันในแนวนอน เช่น แถวไอคอนเมนู แถวรูปภาพสินค้า หรือการ์ดตัวอย่างข้อมูลที่ต้องการให้ผู้ใช้ปัดดูทีละรายการในแนวนอน โดยทั่วไปจะใช้ scrollDirection: Axis.horizontal ร่วมกับ Row เพื่อจัดวางวิดเจ็ตย่อยในแนวนอนก่อนนำมาห่อด้วย SingleChildScrollView

8. อธิบายการใช้ Row ซ้อนใน Column และประโยชน์ในการจัดวางวิดเจ็ตหลายแนวพร้อมกัน

ตอบ ในการออกแบบหน้าจอที่มีโครงร่างซับซ้อน การใช้เพียง Column หรือ Row อย่างใดอย่างหนึ่งอาจไม่เพียงพอ เนื่องจากแต่ละวิดเจ็ตสามารถจัดเรียงองค์ประกอบได้เพียงทิศทางเดียว จึงจำเป็นต้องนำ Row มาซ้อน (nested) ไว้ภายใน children ของ Column เพื่อให้สามารถจัดวางวิดเจ็ตได้ทั้งแนวตั้งและแนวนอนพร้อมกันในหน้าจอเดียว

```
Column(children: [
  Text('Flutter'),
  Text('Flutter is an open source framework...'),
  Row(children: [
    Column(children: [Icon(Icons.thumb_up), Text('130K')]),
    Column(children: [Icon(Icons.share), Text('999')]),
    Column(children: [Icon(Icons.reply), Text('100K')]),
  ]),
])
```

จากตัวอย่างข้างต้น ส่วนหัวข้อและคำอธิบายจัดเรียงในแนวตั้งด้วย Column ส่วนแถวไอคอน Like / Share / Reply ที่ต้องเรียงติดกันในแนวนอนจะใช้ Row ซ้อนอยู่ภายใน Column อีกที และในแต่ละไอคอนที่มีทั้งรูปและข้อความอยู่ด้านล่างก็ยังใช้ Column ซ้อนอยู่ภายใน Row อีกชั้นหนึ่งด้วย

ประโยชน์ของการซ้อน Row ภายใน Column คือช่วยให้สามารถออกแบบโครงร่าง (Layout) ของหน้าจอที่มีความซับซ้อนได้อย่างยืดหยุ่น สามารถผสมผสานการจัดวางวิดเจ็ตได้ทั้งแนวตั้งและแนวนอนในหน้า

เดียวกัน ทำให้ตอบสนองความต้องการของการออกแบบ UI ในสถานการณ์จริงที่มักไม่ได้เป็นเพียงแนวเดียว เช่น หน้าจอโซเชียลมีเดีย หน้าตะกร้าสินค้า หรือหน้าการ์ดแสดงข้อมูลสรุปต่าง ๆ

บทที่7

1. Column Widget **ใช้สำหรับอะไร**

ตอบ Column เป็น multi-child widget ที่ใช้จัดเรียงวิดเจ็ตลูก (child widget) ในแนวตั้ง (vertical) โดยวิดเจ็ตลูกทั้งหมดจะถูกกำหนดผ่านพร็อพเพอร์ตี้ children: ซึ่งมีชนิดข้อมูลเป็น List<Widget> วิดเจ็ตตัวแรกในลิสต์จะถูกวางไว้บนสุดแล้วเรียงลงมาตามลำดับ

2. **ข้อใดคือความหมายของ** Stack Widget

ตอบ คำตอบคือ “วิดเจ็ตทับซ้อนกัน” เนื่องจาก Stack Widget เป็น multi-child widget ที่ใช้จัดเรียงวิดเจ็ตลูกให้ซ้อนทับกันเป็นชั้น ๆ (layer) ตามลำดับที่กำหนดไว้ใน children: ไม่ใช่การจัดเรียงแบบสุ่มหรือแบบเรียงต่อกัน

3. Row Widget **มีแกนหลัก** (Main Axis) **อยู่ในทิศทางใด**

ตอบ แกนหลัก (Main Axis) ของ Row Widget อยู่ในแนวนอน (horizontal) เนื่องจาก Row ใช้จัดเรียงวิดเจ็ตลูกเรียงต่อกันจากซ้ายไปขวา ส่วนแกนตรงข้าม (Cross Axis) ของ Row จะอยู่ในแนวตั้ง

4. Stack Widget **ใช้เพื่อ**

ตอบ ใช้เพื่อจัดเรียงวิดเจ็ตลูกหลายตัวให้วางซ้อนทับกัน (overlapping) โดยสามารถกำหนดตำแหน่งของวิดเจ็ตลูกแต่ละตัวได้ทั้งแบบอิงกับขอบของ Stack (non-positioned ผ่าน alignment:) และแบบกำหนดตำแหน่งอิสระด้วยตนเอง (positioned ผ่าน Positioned widget) เช่น การวางข้อความทับบนรูปภาพ หรือทำปุ่ม Overlay ลอยอยู่บนเนื้อหาอื่น

5. MainAxisAlignment.spaceEvenly **หมายถึง**

ตอบ หมายถึงการกระจายพื้นที่ว่างระหว่างวิดเจ็ตลูกแต่ละตัวให้เท่า ๆ กันตลอดแนวแกนหลัก โดยระยะห่างจากจุดเริ่มต้น (start) ไปยังวิดเจ็ตตัวแรก และระยะห่างจากวิดเจ็ตตัวสุดท้ายไปยังจุดสิ้นสุด (end) จะเท่ากับระยะห่างระหว่างวิดเจ็ตแต่ละตัวด้วยเช่นกัน ต่างจาก spaceAround ที่ระยะขอบจะมีเพียงครึ่งหนึ่งของระยะระหว่างวิดเจ็ต

6. CrossAxisAlignment.stretch **มีผลอย่างไร**

ตอบ มีผลทำให้วิดเจ็ตลูกทุกตัวถูกขยายขนาดให้เต็มพื้นที่ตามแนวแกนตรงข้าม (Cross Axis) เช่น ใน Column วิดเจ็ตลูกจะถูกขยายความกว้างให้เต็มความกว้างของ Column ทำให้ไม่จำเป็นต้องกำหนดความกว้างของแต่ละวิดเจ็ตลูกเอง

7. อธิบายความแตกต่างระหว่าง Column และ Row Widget พร้อมยกตัวอย่างโค้ดการใช้งาน และอธิบายการใช้ VerticalDirection ใน Column Widget และผลลัพธ์ที่ได้เมื่อกำหนดค่าเป็น VerticalDirection.up

ตอบ Column และ Row ต่างก็เป็น multi-child widget ที่ใช้จัดเรียงวิดเจ็ตลูกหลายตัวเหมือนกัน แต่ต่างกันที่ทิศทางของแกนหลัก โดย Column จะจัดเรียงวิดเจ็ตลูกในแนวตั้ง (แกนหลักเป็นแนวตั้ง แกนตรงข้ามเป็นแนวนอน) ส่วน Row จะจัดเรียงวิดเจ็ตลูกในแนวนอน (แกนหลักเป็นแนวนอน แกนตรงข้ามเป็นแนวตั้ง) ตัวอย่างโค้ดการใช้งานทั้งสองวิดเจ็ตมีดังนี้

```
// Column ตัวอย่าง
Column(
  children: <Widget>[
    Container(color: Colors.red, height: 50),
    Container(color: Colors.green, height: 50),
    Container(color: Colors.blue, height: 50),
  ],
),

// Row ตัวอย่าง
Row(
  children: <Widget>[
    Container(color: Colors.red, width: 50),
    Container(color: Colors.green, width: 50),
    Container(color: Colors.blue, width: 50),
  ],
),
```

ตอบ ส่วน VerticalDirection: เป็นพร็อพเพอร์ตี้ที่ใช้กำหนดทิศทางการเรียงวิดเจ็ตลูกภายใน Column ตามแนวตั้ง ค่าเริ่มต้นคือ VerticalDirection.down ซึ่งจะเรียงวิดเจ็ตลูกจากบนลงล่าง โดยวิดเจ็ตตัวแรกจะอยู่ชิดขอบด้านบน หากกำหนดค่าเป็น VerticalDirection.up จะทำให้ลำดับการเรียงกลับกัน คือ วิดเจ็ตตัวแรกจะถูกวางชิดขอบด้านล่างของคอลัมน์ แล้วเรียงวิดเจ็ตตัวถัด ๆ ไปขึ้นไปด้านบนตามลำดับ (ไม่ได้กลับลำดับของ children ในลิสต์ แต่กลับทิศทางการวางตำแหน่งบนหน้าจอ)

8. **อธิบายการใช้ MainAxisAlignment.center ใน Column Widget และผลต่อการจัดวางวิดเจ็ต**

ตอบ MainAxisAlignment.center เป็นการกำหนดจุดเริ่มต้นของการจัดเรียงวิดเจ็ตลูกตามแนวแกนหลักของ Column (คือแนวตั้ง) ให้อยู่กึ่งกลางของแกนหลัก ผลลัพธ์คือวิดเจ็ตลูกทั้งหมดจะถูกจัดกลุ่มและวางอยู่ตรงกลางของพื้นที่ในแนวตั้ง (ไม่ชิดด้านบนหรือด้านล่าง) โดยพื้นที่ว่างด้านบนและด้านล่างของกลุ่มวิดเจ็ตจะเท่ากัน

9. **อธิบายการใช้ SizedBox ในการจัดระยะห่างระหว่างวิดเจ็ต พร้อมยกตัวอย่างโค้ด**

ตอบ SizedBox เป็นวิดเจ็ตที่ใช้กำหนดพื้นที่ว่าง (spacing) ที่มีขนาดคงที่ระหว่างวิดเจ็ตสองตัว นิยมนำไปวางแทรกไว้ระหว่าง children ของ Column หรือ Row เพื่อควบคุมระยะห่างให้แน่นอน โดยกำหนดค่า height: สำหรับระยะห่างในแนวตั้ง (ใช้กับ Column) หรือ width: สำหรับระยะห่างในแนวนอน (ใช้กับ Row) ตัวอย่างการใช้งานมีดังนี้

```
Column(
  children: <Widget>[
    Text('First'),
    SizedBox(height: 15.0),   // ระยะห่างแนวตั้ง 15 พิกเซล
    Text('Second'),
  ],
),
```

10. **อธิบายการใช้ Stack Widget ในการจัดวางวิดเจ็ต พร้อมยกตัวอย่างสถานการณ์ที่เหมาะสม**

ตอบ Stack ใช้จัดวางวิดเจ็ตลูกให้ซ้อนทับกันเป็นชั้น โดยวิดเจ็ตลูกที่อยู่หลังสุดใน children: จะถูกวาดทับอยู่บนสุด สามารถควบคุมตำแหน่งวิดเจ็ตลูกได้สองแบบ คือ non-positioned widget ซึ่งกำหนดตำแหน่งผ่านพร็อพเพอร์ตี้ alignment: ของ Stack และ positioned widget ซึ่งครอบด้วย Positioned widget เพื่อกำหนดตำแหน่ง top, right, bottom, left ได้อย่างอิสระ ตัวอย่างสถานการณ์ที่เหมาะสมกับการใช้ Stack เช่น การวางข้อความบรรยายทับบนรูปภาพพื้นหลัง หรือการทำปุ่ม/ไอคอน Overlay ลอยอยู่มุมหนึ่งของรูปภาพ

Lab 4-6

1. **อธิบายผลกระทบหากแก้ชื่อคลาสหลักใน main.dart แต่ไม่แก้ใน widget_test.dart**

ตอบ หากเปลี่ยนชื่อคลาสหลัก (เช่นจาก MyApp เป็นชื่ออื่น) ในไฟล์ main.dart แต่ไม่ได้แก้ไขชื่อคลาสให้ตรงกันในไฟล์ widget_test.dart จะทำให้เกิดข้อผิดพลาดในการคอมไพล์ (compile error) ประเภท “class not found” หรือ “undefined name” เมื่อรันชุดทดสอบ (flutter test) เนื่องจากไฟล์ widget_test.dart ยังคงอ้างอิงถึงชื่อคลาสเดิมที่ไม่มีอยู่แล้ว ทำให้การทดสอบล้มเหลวทั้งที่โค้ดหลักของแอปพลิเคชันทำงานได้ปกติ ดังนั้นทุกครั้งที่แก้ไขชื่อคลาสหลักจึงต้องแก้ไขให้สอดคล้องกันในทุกไฟล์ที่อ้างอิงถึงคลาสนั้น

2. **อธิบายการใช้ Scaffold และผลที่เกิดขึ้นหากไม่กำหนด AppBar**

ตอบ Scaffold เป็นวิดเจ็ตโครงสร้างพื้นฐานของหน้าจอในแอปพลิเคชัน Flutter ที่ทำตามหลัก Material Design โดยมีพร็อพเพอร์ตี้สำเร็จรูปสำหรับส่วนต่าง ๆ ของหน้าจอ เช่น appBar สำหรับแถบด้านบน body สำหรับเนื้อหาหลัก floatingActionButton สำหรับปุ่มลอย และ drawer สำหรับเมนูด้านข้าง หากไม่กำหนด appBar: ไว้ Scaffold จะไม่แสดงแถบชื่อหน้าจอด้านบน แอปพลิเคชันจะยังคงทำงานได้ปกติไม่เกิดข้อผิดพลาด เพียงแต่พื้นที่ body จะขยายขึ้นไปเต็มพื้นที่ด้านบนของหน้าจอแทน (เหมาะสำหรับหน้าจอที่ไม่ต้องการแถบชื่อ เช่น หน้า Splash Screen)

3. **อธิบายผลลัพธ์หาก Column ไม่กำหนด children**

ตอบ children: เป็นพร็อพเพอร์ตี้ที่จำเป็น (required) ของ Column Widget หากไม่กำหนดค่า children: ให้เลย โปรแกรมจะไม่สามารถคอมไพล์ผ่านได้ และ Dart analyzer จะแจ้งข้อผิดพลาดทันทีว่าพารามิเตอร์ที่จำเป็นขาดหายไป (the parameter 'children' is required) เนื่องจาก Column ต้องมีลิสต์ของวิดเจ็ตลูกอย่างน้อยเพื่อกำหนดว่าจะแสดงผลอะไรบ้าง หากต้องการให้ Column ว่างเปล่าจริง ๆ จะต้องกำหนด children: [] (ลิสต์ว่าง) แทน

4. **อธิบายข้อดีของการแยกคลาสย่อยในการพัฒนาแอปพลิเคชัน**

ตอบ การแยกโค้ดออกเป็นคลาสย่อย (เช่น แยกแต่ละวิดเจ็ตหรือแต่ละหน้าจอออกเป็นคนละคลาส/คนละไฟล์) มีข้อดีหลายประการ ได้แก่ ทำให้โค้ดอ่านง่ายและเป็นระเบียบมากขึ้น ลดความซับซ้อนของไฟล์หลัก ง่ายต่อการดูแลรักษาและแก้ไขโค้ดในภายหลัง สามารถนำคลาสย่อยกลับมาใช้ซ้ำ (reusability) ในส่วนอื่นของแอปพลิเคชันได้ ช่วยให้ทีมพัฒนาสามารถทำงานคู่ขนานกันได้โดยแบ่ง

ความรับผิดชอบเป็นคลาสย่อยที่ชัดเจน และยังง่ายต่อการทดสอบ (unit test) แต่ละส่วนแยกจากกันอีกด้วย

5. อธิบายการทำงานของปุ่ม Random Number และผลลัพธ์ที่ได้

ตอบ ปุ่ม Random Number มักถูกสร้างด้วย ElevatedButton หรือ FloatingActionButton ที่กำหนดฟังก์ชันไว้ในพร็อพเพอร์ตี้ onPressed: เมื่อผู้ใช้กดปุ่ม โปรแกรมจะเรียกใช้คลาส Random จากไลบรารี dart:math เพื่อสุ่มค่าตัวเลข (เช่น Random().nextInt(100)) แล้วนำค่าที่สุ่มได้ไปเก็บไว้ในตัวแปร state และเรียก setState() เพื่อให้ Flutter build วิดเจ็ตใหม่ ผลลัพธ์คือตัวเลขที่แสดงอยู่บนหน้าจอ (เช่นใน Text widget) จะเปลี่ยนเป็นค่าตัวเลขสุ่มใหม่ทุกครั้งที่กดปุ่ม

6. อธิบายผลที่เกิดขึ้นหาก ElevatedButton ไม่กำหนด onPressed

ตอบ หากไม่กำหนดค่าให้พร็อพเพอร์ตี้ onPressed: ของ ElevatedButton หรือกำหนดค่าเป็น null ปุ่มจะยังคงแสดงผลอยู่บนหน้าจอตามปกติ แต่จะอยู่ในสถานะปิดการใช้งาน (disabled state) โดยสีของปุ่มจะจางลงหรือเปลี่ยนเป็นสีเทาตามธีมที่กำหนดไว้ และเมื่อผู้ใช้แตะที่ปุ่มจะไม่มีการตอบสนองใด ๆ เกิดขึ้น เนื่องจาก Flutter ใช้การตรวจสอบว่า onPressed เป็น null หรือไม่เพื่อกำหนดสถานะ enabled/disabled ของปุ่มโดยอัตโนมัติ

7. อธิบายการทำงานของ TextEditingController และประโยชน์ในการควบคุม TextField

ตอบ TextEditingController เป็นคลาสที่ใช้เชื่อมต่อและควบคุมค่าข้อความภายใน TextField หรือ TextFormField โดยเมื่อกำหนดให้กับพร็อพเพอร์ตี้ controller: ของ TextField แล้ว จะสามารถอ่านค่าข้อความปัจจุบันที่ผู้ใช้พิมพ์ได้ผ่าน controller.text กำหนดหรือเปลี่ยนค่าข้อความในช่องกรอกได้จากโค้ด (เช่น เคลียร์ข้อความด้วย controller.clear()) และยังสามารถดักฟังการเปลี่ยนแปลงของข้อความผ่าน addListener() ได้อีกด้วย ประโยชน์คือทำให้นักพัฒนาสามารถควบคุม